Building MQL5-Like Trade Classes in Python for MetaTrader 5

Omega J Msigwa | 26 May, 2025

Contents

- Introduction
- The CAccountInfo class
- The CSymbolInfo class
- The COrderInfo class
- The CHistoryOrderInfo class
- The CPositionInfo class
- The CDealInfo class
- The CTerminalInfo class
- The CTrade class
- Conclusion

Introduction

Building algorithmic trading systems in the MQL5 programming language has been made easier with Standard Libraries that come preloaded in MetaEditor. These modules (libraries) come with functions and variables that simplify the process of opening, validating, closing the trades, etc.

Without these dependencies, it becomes harder to write even a simple program, such as making a simple script for opening a buy position (trade).

Without the CTrade class

```
void OnStart()
  {
   MqlTradeRequest request;
   MqlTradeResult result;

   MqlTick ticks;
   SymbolInfoTick(Symbol(), ticks);

//--- setting a trade request

   ZeroMemory(request);
   request.action   =TRADE_ACTION_DEAL;
   request.symbol   =Symbol();
   request.magic    =2025;
   request.volume   =0.01;
   request.type     =ORDER_TYPE_BUY;
   request.price    =ticks.ask;
   request.sl       =0;
   request.tp       =0;
   request.deviation   = 10;  // Max price slippage in points
   request.magic       = 2025;
   request.comment     = "Buy Order";
   request.type_filling= ORDER_FILLING_IOC;   // or ORDER_FILLING_IOC, ORDER_FILLING_RETURN
   request.type_time   = ORDER_TIME_GTC;      // Good till canceled

//--- action and return the result

   if(!OrderSend(request, result))
    {
      Print("OrderSend failed retcode: ", result.retcode);
    }
  }
```

With the CTrade class

```
#include <Trade\Trade.mqh>
CTrade m_trade;
//+------------------------------------------------------------------+
//| Script program start function                                    |
//+------------------------------------------------------------------+
void OnStart()
  {
//---
    MqlTick ticks;
    SymbolInfoTick(Symbol(), ticks);

    m_trade.SetTypeFillingBySymbol(Symbol());
    m_trade.SetExpertMagicNumber(2025);
    m_trade.SetDeviationInPoints(10); //Slippage
```

```
    m_trade.Buy(0.01, Symbol(), ticks.ask,0,0,"Buy Order");
  }
```

Both these functions open a buy position in MetaTrader 5 but, the first approach is very crude, time-consuming, and it increases the odds of producing bugs due to the large number of lines of code you are supposed to write to get a simple functionality.

Not to mention that it requires you to be more technical (to understand everything that goes into the process of sending a buy position in MetaTrader 5).

There is a Python package known as MetaTrader 5 which gives Python developers access to the platform, an ability to get almost all the information from the platform (symbols, positions opened, etc) and the ability to send some commands for opening, modifying, deleting trades, etc. Similarly to what we can do with the MQL5 programming language.

> As useful as this package is, it doesn't come with built-in modules like those present in the MQL5 language to aid us in the development process.
>
> Similarly to the first coding example, writing a simple program in Python, requires you to write more lines of code and what's even worse is that, this MetaTrade5 Python package doesn't cooperate well with most Integrated Development Environments IDE(s) such as Visual Studio Code, this means that you won't get the Intellisense coding support which is very useful.
>
> Due to the lack of Intellisense in IDE(s) support for this package, you'll often find yourself referring to the documentation for simple concepts you forget instead of figuring them out in the IDE. *This leads to a terrible experience working with this package*.

In this article, we are going to implement the Trade Classes in Python on top of the MetaTrader 5 package to help us write programs effectively in Python as in MQL5.

The CAccountInfo Class

In MQL5 this class is for working with trade account properties. All the information about the trading account on a broker can be accessed using this class, let's make its equivalent in Python.

Python Custom CAccountInfo trade class	MQL5 Built-in CAccountInfo trade class	Description
Integer & String type properties		
`login()`	Login	Gets the account number.
`trade_mode()`	TradeMode	Gets the trade mode.
`trade_mode_description()`	TradeModeDescription	Gets the trade mode as a string.
`leverage()`	Leverage	Gets the amount of given leverage.
`stopout_mode()`	StopoutMode	Gets the mode of stop out setting.
`stopout_mode_description()`	StopoutModeDescription	Gets the mode of stop out settting as a string.
`margin_mode()`	MarginMode	Gets the margin calculation mode.
`margin_mode_description()`	MarginModeDescription	Gets the margin calculation mode as a string.
`trade_allowed()`	TradeAllowed	Gets the flag of trade allowance.

Python Custom CAccountInfo trade class	MQL5 Built-in CAccountInfo trade class	Description
`trade_expert()`	TradeExpert	Gets the flag of automated trade allowance.
`limit_orders()`	LimitOrders	Gets the maximal number of allowed pending orders.
Double type properties		
`balance()`	Balance	Gets the balance of MetaTrader 5 account.
`credit()`	Credit	Gets the amount of given credit.
`profit()`	Profit	Gets the amount of current profit on an account
`equity()`	Equity	Gets the amount of current equity on account.
`margin()`	Margin	Gets the amount of reserved margin.
`free_margin()`	FreeMargin	Gets the amount of free margin.
`margin_level()`	MarginLevel	Gets the level of margin.
`margin_call()`	MarginCall	Gets the level of margin for deposit.
`margin_stopout()`	MarginStopOut	Gets the level of margin for stop out.
Text type properties		
`name()`	Name	Gets the account name
`server()`	Server	Gets the trade server name
`company()`	Company	Gets the company name that serves the account
`currency()`	Currency	Gets the deposit

Python Custom CAccountInfo trade class	MQL5 Built-in CAccountInfo trade class	Description
		currency name.
Additional methods		
`margin_check(self, symbol, order_type, volume, price)`	MarginCheck	Gets the amount of margin required to execute trade operation.
`free_margin_check(self, symbol, order_type, volume, price)`	FreeMarginCheck	Gets the amount of free margin left after execution of trade operation.
`order_profit_check(self, symbol, order_type, volume, price_open, price_close)`	OrderProfitCheck	Gets the avaluated profit based on the parameters passed.
`max_lot_check(self, symbol, order_type, price, percent=100)`	MaxLotCheck	Gets the maximal possible volume of trade operation.

Example usage

```
import MetaTrader5 as mt5
from Trade.AccountInfo import CAccountInfo

if not mt5.initialize(r"c:\Users\Omega Joctan\AppData\Roaming\Pepperstone MetaTrader 5\terminal64.exe"):
    print("Failed to initialize Metatrader5 Error = ",mt5.last_error())
    quit()

acc = CAccountInfo()

print(f"""
Account Information
--------------------
Login: {acc.login()}
Name: {acc.name()}
Server: {acc.server()}
Company: {acc.company()}
Currency: {acc.currency()}
Trade Mode: {acc.trade_mode()} ({acc.trade_mode_description()})
Leverage: {acc.leverage()}
Stopout Mode: {acc.stopout_mode()} ({acc.stopout_mode_description()})
Margin Mode: {acc.margin_mode()} ({acc.margin_mode_description()})
Trade Allowed: {acc.trade_allowed()}
Trade Expert: {acc.trade_expert()}
Limit Orders: {acc.limit_orders()}
--------------------
Balance: {acc.balance()}
Credit: {acc.credit()}
Profit: {acc.profit()}
Equity: {acc.equity()}
Margin: {acc.margin()}
Free Margin: {acc.free_margin()}
Margin Level: {acc.margin_level()}
Margin Call: {acc.margin_call()}
Margin StopOut: {acc.margin_stopout()}
--------------------
""")

mt5.shutdown()
```

Outputs

```
Account Information
-------------------
Login: 61346344
Name: John Doe
Server: MetaQuotes-Demo
Company: MetaQuotes Software Corp
Currency: USD
Trade Mode: 0 (Demo)
Leverage: 400
Stopout Mode: 0 (Percent)
Margin Mode: 2 (Retail Hedging)
Trade Allowed: True
Trade Expert: True
Limit Orders: 500
-------------------
Balance: 928.42
Credit: 0.0
Profit: -2.21
Equity: 926.21
Margin: 2.81
Free Margin: 923.4
Margin Level: 32961.20996441281
Margin Call: 90.0
Margin StopOut: 20.0
-------------------
```

The CSymbolInfo Class

This class provides access to the symbol properties.

Python custom CSymbolInfo class	MQL5 built-in CSymbolInfo class	Description
Controlling		
`refresh()`	Refresh	Refreshes the symbol data.
`refresh_rates()`	RefreshRates	Refreshes the symbol quotes
Properties		
`name()`	Name	Gets the symbol name.
`select(self, select=True)`	Select	Adds or removes the symbol to and from the "Market Watch"
`is_synchronized()`	IsSynchronized	Checks the symbol synchronization with the server.
Volumes		
`volume()`	Volume	Gets the volumne of the last deal.
`volume_high()`	VolumeHigh	Gets the maximal volume for a day
`volume_low()`	VolumeLow	Gets the minimal volume for a day.
Miscellaneous		
`time()`	Time	Gets the time of last quote.
`spread()`	Spread	Gets the amount of spread (in points).
`spread_float()`	SpreadFloat	Gets the flag of floating spread.
`ticks_book_depth()`	TicksBookDepth	Gets the depth of ticks saving.

Python custom CSymbolInfo class	MQL5 built-in CSymbolInfo class	Description
Levels		
`stops_level()`	StopsLevel	Gets the minimal indent for orders (in points).
`freeze_level()`	FreezeLevel	Gets the distance of freezing trade operations (in points).
Bid prices		
`bid()`	Bid	Gets the current bid price.
`bid_high()`	BidHigh	Gets the maximal bid price for a day.
`bid_low()`	BidLow	Gets the minimal bid price for a day.
Ask prices		
`ask()`	Ask	Gets the current Ask price
`ask_high()`	AskHigh	Gets the maximal Ask price for a day
`ask_low()`	AskLow	Gets the minimal Ask price for a day
Prices		
`last()`	Last	Returns the current last price
`last_high()`	LastHigh	Returns the maximal last price for a day
`last_low()`	LastLow	Returns the minimal last price for a day
Trade modes		
`trade_calc_mode()`	TradeCalcMode	Gets the mode of contract cost calculation in integer format.
`trade_calc_mode_description()`	TradeCalcModeDescription	Gets the mode of contract cost calculation in string format.
`trade_mode()`	TradeMode	Gets the type of order execution in integer format.
`trade_mode_description()`	TradeModeDescription	Gets the type of order execution in string format.
`trade_execution()`	TradeExecution	Gets the trade execution mode in integer format.
`trade_execution_description()`	TradeExecutionDescription	Gets the trade execution mode in string format
Swaps		
`swap_mode()`	SwapMode	Gets the swap calculation mode in integer format
`swap_mode_description()`	SwapModeDescription	Gets the swap calculation mode in string format
`swap_rollover_3days()`	SwapRollover3days	Gets the day of triple swap charge as an integer
`swap_rollover_3days_description()`	SwapRollover3daysDescription	Gets the day of triple swap charge as a string.
Margin		
`margin_initial()`	MarginInitial	Gets the value of initial margin

Python custom CSymbolInfo class	MQL5 built-in CSymbolInfo class	Description
`margin_maintenance()`	MarginMaintenance	Gets the value of maintenance margin
`margin_hedged()`		Returns the hedged margin value for the given symbol.
`margin_hedged_use_leg()`		Returns a boolean that tells whether the hedged margin applies to each leg (position side) individually.
Tick information		
`digits()`	Digits	Gets the number of digits after period
`point()`	Point	Gets the value of one point
`tick_value()`	TickValue	Gets the tick value (minimal change of price)
`tick_value_profit()`	TickValueProfit	Gets the calculated tick price of a profitable position
`tick_value_loss()`	TickValueLoss	Gets the calculated tick price for a losing position
`tick_size()`	TickSize	Gets the minimal change of price
Contracts sizes		
`contract_size()`	ContractSize	Gets the amount of trade contract
`lots_min()`	LotsMin	Gets the minimal volume to close a deal
`lots_max()`	LotsMax	Gets the maximal volume to close a deal
`lots_step()`	LotsStep	Gets the minimal step of volume change to close a deal
`lots_limit()`	LotsLimit	Gets the maximal allowed volume of opened position and pending orders in either direction for one symbol
Swap sizes		
`swap_long()`	SwapLong	Gets the value of long position swap
`swap_short()`	SwapShort	Gets the value of short position swap
Symbol/Currency Information		
`currency_base()`	CurrencyBase	Gets the name of symbol base currency
`currency_profit()`	CurrencyProfit	Gets the profit currency name
`currency_margin()`	CurrencyMargin	Gets the margin currency name
`bank()`	Bank	Gets the name of the current quote source
`description()`	Description	Gets the string description of a symbol
`path()`	Path	Gets the path in symbols tree
`page()`		The address of a webpage containing symbol's information
Session Information		
`session_deals()`	SessionDeals	Gets the number of deals in the current session
`session_buy_orders()`	SessionBuyOrders	Gets the number of buy orders presently

Python custom CSymbolInfo class	MQL5 built-in CSymbolInfo class	Description
`session_sell_orders()`	SessionSellOrders	Gets the number of sell orders presently
`session_turnover()`	SessionTurnover	Gets the summary of turnover of the current session
`session_interest()`	SessionInterest	Gets the summary of open interest of the current session
`session_buy_orders_volume()`	SessionBuyOrdersVolume	Gets the volume of buy orders
`session_sell_orders_volume()`	SessionSellOrdersVolume	Gets the volume of sell orders
`session_open()`	SessionOpen	Gets the open price of the current session
`session_close()`	SessionClose	Gets the close price of the current session
`session_aw()`	SessionAW	Gets the average weighted price of the current session
`session_price_settlement()`	SessionPriceSettlement	Gets the settlement price of the current session
`session_price_limit_min()`	SessionPriceLimitMin	Gets the minimal price of the current session
`session_price_limit_max()`	SessionPriceLimitMax	Gets the maximal price of the current session

These are some of the methods in the Python class, a full list can be seen inside the file SymbolInfo.py.

Example Usage

```
import MetaTrader5 as mt5
from Trade.SymbolInfo import CSymbolInfo


if not mt5.initialize(r"c:\Users\Omega Joctan\AppData\Roaming\Pepperstone MetaTrader 5\terminal64.exe"):
    print("Failed to initialize Metatrader5 Error = ",mt5.last_error())
    quit()

m_symbol = CSymbolInfo("EURUSD")

print(f"""
Symbol Information
---------------------
Name: {m_symbol.name()}
Selected: {m_symbol.select()}
Synchronized: {m_symbol.is_synchronized()}

--- Volumes ---
Volume: {m_symbol.volume()}
Volume High: {m_symbol.volume_high()}
Volume Low: {m_symbol.volume_low()}

--- Time & Spread ---
Time: {m_symbol.time()}
Spread: {m_symbol.spread()}
Spread Float: {m_symbol.spread_float()}
Ticks Book Depth: {m_symbol.ticks_book_depth()}

--- Trade Levels ---
Stops Level: {m_symbol.stops_level()}
Freeze Level: {m_symbol.freeze_level()}

--- Bid Parameters ---
Bid: {m_symbol.bid()}
Bid High: {m_symbol.bid_high()}
Bid Low: {m_symbol.bid_low()}

--- Ask Parameters ---
Ask: {m_symbol.ask()}
Ask High: {m_symbol.ask_high()}
Ask Low: {m_symbol.ask_low()}

--- Last Parameters ---
Last: {m_symbol.last()}
Last High: {m_symbol.last_high()}
Last Low: {m_symbol.last_low()}

--- Order & Trade Modes ---
```

```
Trade Calc Mode: {m_symbol.trade_calc_mode()} ({m_symbol.trade_calc_mode_description()})
Trade Mode: {m_symbol.trade_mode()} ({m_symbol.trade_mode_description()})
Trade Execution Mode: {m_symbol.trade_execution()}  ({m_symbol.trade_execution_description()})

--- Swap Terms ---
Swap Mode: {m_symbol.swap_mode()} ({m_symbol.swap_mode_description()})
Swap Rollover 3 Days: {m_symbol.swap_rollover_3days()} ({m_symbol.swap_rollover_3days_description()})

--- Futures Dates ---
Start Time: {m_symbol.start_time()}
Expiration Time: {m_symbol.expiration_time()}

--- Margin Parameters ---
Initial Margin: {m_symbol.margin_initial()}
Maintenance Margin: {m_symbol.margin_maintenance()}
Hedged Margin: {m_symbol.margin_hedged()}
Hedged Margin Use Leg: {m_symbol.margin_hedged_use_leg()}

--- Tick Info ---

Digits: {m_symbol.digits()}
Point: {m_symbol.point()}
Tick Value: {m_symbol.tick_value()}
Tick Value Profit: {m_symbol.tick_value_profit()}
Tick Value Loss: {m_symbol.tick_value_loss()}
Tick Size: {m_symbol.tick_size()}

--- Contracts sizes---
Contract Size: {m_symbol.contract_size()}
Lots Min: {m_symbol.lots_min()}
Lots Max: {m_symbol.lots_max()}
Lots Step: {m_symbol.lots_step()}
Lots Limit: {m_symbol.lots_limit()}

--- Swap sizes

Swap Long: {m_symbol.swap_long()}
Swap Short: {m_symbol.swap_short()}

--- Currency Info ---
Currency Base: {m_symbol.currency_base()}
Currency Profit: {m_symbol.currency_profit()}
Currency Margin: {m_symbol.currency_margin()}
Bank: {m_symbol.bank()}
Description: {m_symbol.description()}
Path: {m_symbol.path()}
Page: {m_symbol.page()}

--- Session Info ---
Session Deals: {m_symbol.session_deals()}
Session Buy Orders: {m_symbol.session_buy_orders()}
Session Sell Orders: {m_symbol.session_sell_orders()}
Session Turnover: {m_symbol.session_turnover()}
Session Interest: {m_symbol.session_interest()}
Session Buy Volume: {m_symbol.session_buy_orders_volume()}
Session Sell Volume: {m_symbol.session_sell_orders_volume()}
Session Open: {m_symbol.session_open()}
Session Close: {m_symbol.session_close()}
Session AW: {m_symbol.session_aw()}
Session Price Settlement: {m_symbol.session_price_settlement()}
Session Price Limit Min: {m_symbol.session_price_limit_min()}
Session Price Limit Max: {m_symbol.session_price_limit_max()}
---------------------
""")

mt5.shutdown()
```

Outputs

```
Symbol Information
---------------------
Name: EURUSD
Selected: True
Synchronized: True

--- Volumes ---
Volume: 0
Volume High: 0
Volume Low: 0

--- Time & Spread ---
Time: 2025-05-21 20:30:36
```

```
Spread: 0
Spread Float: True
Ticks Book Depth: 0

--- Trade Levels ---
Stops Level: 0
Freeze Level: 0

--- Bid Parameters ---
Bid: 1.1335600000000001
Bid High: 1.13623
Bid Low: 1.12784

--- Ask Parameters ---
Ask: 1.1335600000000001
Ask High: 1.13623
Ask Low: 1.12805

--- Last Parameters ---
Last: 0.0
Last High: 0.0
Last Low: 0.0

--- Order & Trade Modes ---
Trade Calc Mode: 0 (Calculation of profit and margin for Forex)
Trade Mode: 4 (No trade restrictions)
Trade Execution Mode: 2  (Market execution)

--- Swap Terms ---
Swap Mode: 1 (Swaps are calculated in points)
Swap Rollover 3 Days: 3 (Wednesday)

--- Futures Dates ---
Start Time: 0
Expiration Time: 0

--- Margin Parameters ---
Initial Margin: 100000.0
Maintenance Margin: 0.0
Hedged Margin: 0.0
Hedged Margin Use Leg: False

--- Tick Info ---

Digits: 5
Point: 1e-05
Tick Value: 1.0
Tick Value Profit: 1.0
Tick Value Loss: 1.0
Tick Size: 1e-05

--- Contracts sizes---
Contract Size: 100000.0
Lots Min: 0.01
Lots Max: 100.0
Lots Step: 0.01
Lots Limit: 0.0

--- Swap sizes

Swap Long: -8.99
Swap Short: 4.5

--- Currency Info ---
Currency Base: EUR
Currency Profit: USD
Currency Margin: EUR
Bank: Pepperstone
Description: Euro vs US Dollar
Path: Markets\Forex\Majors\EURUSD
Page:

--- Session Info ---
Session Deals: 1
Session Buy Orders: 647
Session Sell Orders: 2
Session Turnover: 10.0
Session Interest: 0.0
Session Buy Volume: 3.0
Session Sell Volume: 13.0
Session Open: 1.12817
Session Close: 1.12842
Session AW: 0.0
Session Price Settlement: 0.0
```

```
Session Price Limit Min: 0.0
Session Price Limit Max: 0.0
---------------------
```

The COrderInfo Class

This class provides access to the pending order properties.

Python custom COrderInfo class	MQL5 built-in COrderInfo class	Description
Integer & datetime type properties		
`ticket()`	Ticket	Gets the ticket of an order, previously selected for access.
`type_time()`	TypeTime	Gets the type of order at the time of the expiration.
`type_time_description()`	TypeTimeDescription	Gets the order type by expiration as a string
`time_setup()`	TimeSetup	Gets the time of order placement.
`time_setup_msc()`	TimeSetupMsc	Receives the time of placing an order in milliseconds since 01.01.1970.
`order_type()`	OrderType	Gets the order type in integer format.
`order_type_description()`	OrderTypeDescription	Gets the order type as a string
`state()`	State	Gets the order state as an integer.
`state_description()`	StateDescription	Gets the order state as a string.
`magic()`	Magic	Gets the ID of the expert that placed the order.
`position_id()`	PositionId	Gets the ID of position.
`type_filling()`	TypeFilling	Gets the type of order execution by remainder as an integer.
`type_filling_description()`	TypeFillingDescription	Gets the type of order execution by remainder as a string.
`time_done()`	TimeDone	Gets the time of order execution or cancellation.
`time_done_msc()`	TimeDoneMsc	Receives order execution or cancellation time in milliseconds since since 01.01.1970.
`time_expiration()`	TimeExpiration	Gets the time of order expiration.
Double type properties		
`volume_initial()`	VolumeInitial	Gets the initial volume of order.
`volume_current()`	VolumeCurrent	Gets the unfilled volume of order.
`price_open()`	PriceOpen	Gets the order price.
`price_current()`	PriceCurrent	Gets the current price by order symbol.
`stop_loss()`	StopLoss	Gets the order's Stop loss.
`take_profit()`	TakeProfit	Gets the order's Take profit.
`price_stop_limit()`	PriceStopLimit	Gets the price of a limit order.
Access to text properties		

Python custom COrderInfo class	MQL5 built-in COrderInfo class	Description
`comment()`	Symbol	Gets the order comment.
`symbol()`	Comment	Gets the name of the order symbol.
Selection		
`select_order(self, order) -> bool`		Selects an order by its object (dictionary) from list of oders returned by the function MetaTrader5.orders_get()

Example Usage

```
import MetaTrader5 as mt5
from Trade.OrderInfo import COrderInfo

if not mt5.initialize(r"c:\Users\Omega Joctan\AppData\Roaming\Pepperstone MetaTrader 5\terminal64.exe"):
    print("Failed to initialize Metatrader5 Error = ",mt5.last_error())
    quit()

# Get all orders from MT5
orders = mt5.orders_get()

# Loop and print info
m_order = COrderInfo()

for i, order in enumerate(orders):
    if m_order.select_order(order=order):
        print(f"""
Order #{i}

--- Integer & datetime type properties ---

Ticket: {m_order.ticket()}
Type Time: {m_order.type_time()} ({m_order.type_time_description()})
Time Setup: {m_order.time_setup()}
Time Setup (ms): {m_order.time_setup_msc()}
State: {m_order.state()} ({m_order.state_description()})
Order Type: {m_order.order_type()} ({m_order.order_type_description()})
Magic Number: {m_order.magic()}
Position ID: {m_order.position_id()}
Type Filling: {m_order.type_filling()} ({m_order.type_filling_description()})
Time Done: {m_order.time_done()}
Time Done (ms): {m_order.time_done_msc()}
Time Expiration: {m_order.time_expiration()}
External ID: {m_order.external_id()}

--- Double type properties ---

Volume Initial: {m_order.volume_initial()}
Volume Current: {m_order.volume_current()}
Price Open: {m_order.price_open()}
Price Current: {m_order.price_current()}
Stop Loss: {m_order.stop_loss()}
Take Profit: {m_order.take_profit()}
Price StopLimit: {m_order.price_stop_limit()}

--- Text type properties ---

Comment: {m_order.comment()}
Symbol: {m_order.symbol()}

""")

mt5.shutdown()
```

Outputs

```
Order #0

--- Integer & datetime type properties ---

Ticket: 153201235
Type Time: 2 (ORDER_TIME_SPECIFIED)
Time Setup: 2025-05-21 23:56:16
Time Setup (ms): 1747860976672
State: 1 (Order accepted)
Order Type: 3 (Sell Limit pending order)
```

```
Magic Number: 1001
Position ID: 0
Type Filling: 2 (IOC (Immediate or Cancel))
Time Done: 1970-01-01 03:00:00
Time Done (ms): 0
Time Expiration: 2025-05-21 23:57:14.940000
External ID:

--- Double type properties ---

Volume Initial: 0.01
Volume Current: 0.01
Price Open: 1.13594
Price Current: 1.1324
Stop Loss: 0.0
Take Profit: 0.0
Price StopLimit: 0.0

--- Text type properties ---

Comment: Sell Limit Order
Symbol: EURUSD
```

The CHistoryOrderInfo Class

This class provides easy access to the history order properties.

Python custom CHistoryOrderInfo class	MQL5 built-in CHistoryOrderInfo class	Description
Integer, Datetime, and String type properties		
`time_setup()`	TimeSetup	Gets the time of order placement.
`time_setup_msc()`	TimeSetupMsc	Returns the time of placing an order in milliseconds since 01.01.1970
`time_done()`	TimeDone	Gets the time of order execution or cancellation.
`time_done_msc()`	TimeDoneMsc	Returns order execution or cancellation time in milliseconds since 01.01.1970
`magic()`	Magic	Gets the ID of an expert advisor that placed a selected order
`ticket()`		Returns the ticket of the selected order.
`order_type()`	OrderType	Returns the type of the selected order.
`order_type_description()`	OrderTypeDescription	Returns the type of the selected order as a string
`state()`	State	Returns the order state as an integer.
`state_description()`	StateDescription	Returns the order state as a string.
`time_expiration()`	TimeExpiration	Gets the time of the selected order expiration.
`type_filling()`	TypeFilling	Gets the type of order execution by remainder in integer format.
`type_filling_description()`	TypeFillingDescription	Gets the type of order execution by remainder as a string.
`type_time()`	TypeTime	Gets the type of the selected order type at the time of the expiration as an integer.
`type_time_description()`	TypeTimeDescription	Gets the selected order type at the time of the expiration in string format.
`position_id()`	PositionId	Gets the position ID
Double type properties		
`volume_initial()`	VolumeInitial	Gets the initial volume of the selected order

Python custom CHistoryOrderInfo class	MQL5 built-in CHistoryOrderInfo class	Description
`volume_current()`	VolumeCurrent	Gets the unfufilled volume of the selected order.
`price_open()`	PriceOpen	Gets the selected order price.
`price_current()`	PriceCurrent	Gets the current price by order symbol.
`stop_loss()`	StopLoss	Gets the selected order's stop loss.
`take_profit()`	TakeProfit	Gets the selected order's take profit.
`price_stop_limit()`	PriceStopLimit	Gets the price of a selected limit order.
Text properties		
`symbol()`	Symbol	Returns the symbol of a selected order.
`comment()`	Comment	Returns the comment of a selected order.
Selection		
`select_order(self, order) -> bool`		Selects an order by it's object from a list of objects (dictionaries) returned from the function MetaTrader5.history_orders_get.

Example usage

```
import MetaTrader5 as mt5
from Trade.HistoryOrderInfo import CHistoryOrderInfo
from datetime import datetime, timedelta

if not mt5.initialize(r"c:\Users\Omega Joctan\AppData\Roaming\Pepperstone MetaTrader 5\terminal64.exe"):
    print("Failed to initialize Metatrader5 Error = ",mt5.last_error())
    quit()

# The date range

from_date = datetime.now() - timedelta(hours=5)
to_date = datetime.now()

# Get history orders

history_orders = mt5.history_orders_get(from_date, to_date)

if history_orders == None:
    print(f"No deals, error code={mt5.last_error()}")
    exit()

# m_order instance
m_order = CHistoryOrderInfo()

# Loop and print each order
for i, order in enumerate(history_orders):
    if m_order.select_order(order):
        print(f"""
History Order #{i}

--- Integer, Datetime & String type properties ---

Time Setup: {m_order.time_setup()}
Time Setup (ms): {m_order.time_setup_msc()}
Time Done: {m_order.time_done()}
Time Done (ms): {m_order.time_done_msc()}
Magic Number: {m_order.magic()}
Ticket: {m_order.ticket()}
Order Type: {m_order.order_type()} ({m_order.type_description()})
Order State: {m_order.state()} ({m_order.state_description()})
Expiration Time: {m_order.time_expiration()}
Filling Type: {m_order.type_filling()} ({m_order.type_filling_description()})
Time Type: {m_order.type_time()} ({m_order.type_time_description()})
Position ID: {m_order.position_id()}
Position By ID: {m_order.position_by_id()}
```

```
--- Double type properties ---

Volume Initial: {m_order.volume_initial()}
Volume Current: {m_order.volume_current()}
Price Open: {m_order.price_open()}
Price Current: {m_order.price_current()}
Stop Loss: {m_order.stop_loss()}
Take Profit: {m_order.take_profit()}
Price Stop Limit: {m_order.price_stop_limit()}

--- Access to text properties ---

Symbol: {m_order.symbol()}
Comment: {m_order.comment()}
""")

mt5.shutdown()
```

Outputs

```
History Order #79

--- Integer, Datetime & String type properties ---

Time Setup: 2025-05-21 23:56:17
Time Setup (ms): 1747860977335
Time Done: 2025-05-22 01:57:47
Time Done (ms): 1747868267618
Magic Number: 1001
Ticket: 153201241
Order Type: 5 (Sell Stop pending order)
Order State: 4 (Order fully executed)
Expiration Time: 2025-05-21 23:57:14.940000
Filling Type: 1 (FOK (Fill or Kill))
Time Type: 2 (ORDER_TIME_SPECIFIED)
Position ID: 153201241
Position By ID: 0

--- Double type properties ---

Volume Initial: 0.01
Volume Current: 0.0
Price Open: 1.13194
Price Current: 1.13194
Stop Loss: 0.0
Take Profit: 0.0
Price Stop Limit: 0.0

--- Access to text properties ---

Symbol: EURUSD
Comment: Sell Stop Order
```

The CPositionInfo Class

This class provides easy access to the open position properties.

Python custom CPositionInfo class	MQL5 built-in CPositionInfo class	Description
Integer & datetime type properties		
`ticket()`		Gets the ticket of a position, previously selected for access.
`time()`	Time	Gets the time of position opening.
`time_msc()`	TimeMsc	Receives the time of placing a position in milliseconds since 01.01.1970.
`time_update()`	TimeUpdate	Receives the time of position changing in seconds since 01.01.1970.
`time_update_msc()`	TimeUpdateMsc	Receives the time of position changing in milliseconds since 01.01.1970.
`position_type()`	PositionType	Gets the position type as an integer.

Python custom CPositionInfo class	MQL5 built-in CPositionInfo class	Description
`position_type_description()`	TypeDescription	Gets the position type as a string
`magic()`	Magic	Gets the ID of the expert that opened the position.
`position_id()`	Identifier	Gets the ID of position.
Double type properties		
`volume()`	Volume	Gets the volume of position.
`price_open()`	PriceOpen	Gets the price of position opening.
`stop_loss()`	StopLoss	Gets the price of position's Stop loss.
`take_profit()`	TakeProfit	Gets the price of position's Take profit.
`price_current()`	PriceCurrent	Gets the current price by position symbol.
`profit()`	Profit	Gets the amount of current profit by position.
`swap()`	Swap	Gets the amount of swap by position.
Access to text properties		
`comment()`	Comment	Gets the comment of the position.
`symbol()`	Symbol	Gets the name of position symbol.
Selection		
`select_position(self, position) -> bool`		Selects the position object (dictionary) from a list of positions returned by the function MetaTrader5.positions_get()

Example Usage

```
import MetaTrader5 as mt5
from Trade.PositionInfo import CPositionInfo

if not mt5.initialize(r"c:\Users\Omega Joctan\AppData\Roaming\Pepperstone MetaTrader 5\terminal64.exe"):
    print("Failed to initialize Metatrader5 Error = ",mt5.last_error())
    quit()

positions = mt5.positions_get()
m_position = CPositionInfo()

# Loop and print each position
for i, position in enumerate(positions):
    if m_position.select_position(position):
        print(f"""
Position #{i}

--- Integer type properties ---

Time Open: {m_position.time()}
Time Open (ms): {m_position.time_msc()}
Time Update: {m_position.time_update()}
Time Update (ms): {m_position.time_update_msc()}
Magic Number: {m_position.magic()}
Ticket: {m_position.ticket()}
Position Type: {m_position.position_type()} ({m_position.position_type_description()})

--- Double type properties ---

Volume: {m_position.volume()}
Price Open: {m_position.price_open()}
Price Current: {m_position.price_current()}
Stop Loss: {m_position.stop_loss()}
Take Profit: {m_position.take_profit()}
```

```
Profit: {m_position.profit()}
Swap: {m_position.swap()}

--- Access to text properties ---

Symbol: {m_position.symbol()}
Comment: {m_position.comment()}

""")

mt5.shutdown()
```

Outputs

```
Position #1

--- Integer type properties ---

Time Open: 2025-05-22 15:02:06
Time Open (ms): 1747915326225
Time Update: 2025-05-22 15:02:06
Time Update (ms): 1747915326225
Magic Number: 0
Ticket: 153362497
Position Type: 1 (Sell)

--- Double type properties ---

Volume: 0.1
Price Open: 1.12961
Price Current: 1.1296
Stop Loss: 0.0
Take Profit: 0.0
Profit: 0.1
Swap: 0.0

--- Access to text properties ---

Symbol: EURUSD
Comment:
```

The CDealInfo Class

This class provides access to the deal properties from the MetaTrader 5 program.

Python custom CDealInfo class	MQL5 built-in CDealInfo class	Description
Interger and datetime type properties		
`ticket()`		Gives the ticket of a selected deal
`time()`	Time	Gets the time of deal execution.
`time_msc()`	TimeMsc	Receives the time of a deal execution in milliseconds since 01.01.1970
`deal_type()`	DealType	Gets the deal type
`type_description()`	TypeDescription	Gets the deal type as a string.
`entry()`	Entry	Gets the deal direction
`entry_description()`	EntryDescription	Gets the deal direction as a string.
`magic()`	Magic	Gets the ID of the expert, that executed the deal.
`position_id()`	PositionId	Gets the ID of the position, in which the deal was involved.
Double type properties		

Python custom CDealInfo class	MQL5 built-in CDealInfo class	Description
`volume()`	Volume	Gets the volume (lot size) of the deal.
`price()`	Price	Gets the deal price.
`commission()`	Commision	Gets the commission of the deal.
`swap()`	Swap	Gets the amount of swap when the position is closed
`profit()`	Profit	Gets the financial result (profit) of the deal
String type properties		
`symbol()`	Symbol	Gets the name of the selected deal symbol.
`comment()`	Comment	Gets the comment of the selected deal.
Selection		
`select_by_index(self, index)`		Selects the deal by index.
`select_deal(self, deal) -> bool`		Selects a deal by its object (dictionary) from list of deals returned by the function MetaTrader5.history_deals_get

Example usage

```
import MetaTrader5 as mt5
from datetime import datetime, timedelta
from Trade.DealInfo import CDealInfo

# The date range
from_date = datetime.now() - timedelta(hours=24)
to_date = datetime.now()

if not mt5.initialize(r"c:\Users\Omega Joctan\AppData\Roaming\Pepperstone MetaTrader 5\terminal64.exe"):
    print("Failed to initialize Metatrader5 Error = ",mt5.last_error())
    quit()

m_deal = CDealInfo()

# Get all deals from MT5 history
deals = mt5.history_deals_get(from_date, to_date)

for i, deal in enumerate(deals):

    if (m_deal.select_deal(deal=deal)):
        print(f"""
Deal #{i}

--- integer and dateteime properties ---

Ticket: {m_deal.ticket()}
Time: {m_deal.time()}
Time (ms): {m_deal.time_msc()}
Deal Type: {m_deal.deal_type()} ({m_deal.type_description()})
Entry Type: {m_deal.entry()} ({m_deal.entry_description()})
Order: {m_deal.order()}
Magic Number: {m_deal.magic()}
Position ID: {m_deal.position_id()}

--- double type properties ---

Volume: {m_deal.volume()}
Price: {m_deal.price()}
Commission: {m_deal.commission()}
Swap: {m_deal.swap()}
Profit: {m_deal.profit()}

--- string type properties ---

Comment: {m_deal.comment()}
Symbol: {m_deal.symbol()}

External ID: {m_deal.external_id()}
```

```
""")

mt5.shutdown()
```

Outputs

```
Deal #53

--- integer and dateteime properties ---

Ticket: 0
Time: 2025-05-22 01:57:47
Time (ms): 1747868267618
Deal Type: 1 (SELL)
Entry Type: 0 (IN)
Order: 153201241
Magic Number: 1001
Position ID: 153201241

--- double type properties ---

Volume: 0.01
Price: 1.13194
Commission: -0.04
Swap: 0.0
Profit: 0.0

--- string type properties ---

Comment: Sell Stop Order
Symbol: EURUSD

External ID:
```

The CTerminalInfo Class

This class provides access to the properties of the MetaTrader 5 program environment.

Python custom CTerminalInfo class	MQL5 built-in CTerminalInfo class	Description
String type properties		
`name()`	Name	Gets the name of the client terminal.
`company()`	Company	Gets the company name of the client terminal.
`language()`	Language	Gets the language of the client terminal.
`path()`	Path	Gets the folder of the client terminal.
`data_path()`	DataPath	Gets the data folder for the client terminal.
`common_data_path()`	CommonDataPath	Gets the common data folder of all client terminals (All MetaTrade5 apps installed on the computer.
Integer type properties		
`build()`	Build	Gets the build number of the client terminal.
`is_connected()`	IsConnected	Gets the information about connection to trade server.
`is_dlls_allowed()`	IsDLLsAllowed	Gets the information about permission of DLL usage.
`is_trade_allowed()`	IsTradeAllowed	Gets the information about permission to trade.
`is_email_enabled()`	IsEmailEnabled	Gets the information about permission to send e-mails to SMTP server and login, specified in the terminal settings.
`is_ftp_enabled()`	IsFtpEnabled	Gets the information about permission to send trade reports to FTP server and login, specified in the terminal settings.

Python custom CTerminalInfo class	MQL5 built-in CTerminalInfo class	Description
`are_notifications_enabled()`		Checks whether push notifications are enabled in MetaTrader 5 terminal settings.
`is_community_account()`		Checks if the current terminal is logged into a MetaTrader community in mql5.com (this website)
`is_community_connection()`		Checks if the terminal has an active connection to the MQL5 community services.
`is_mqid()`		Checks if the user is signed in using their MQID (MetaQuotes ID).
`is_tradeapi_disabled()`		Checks if the Trade API is disabled in MetaTrader 5 settings.
`max_bars()`	MaxBars	Gets the information about the maximum number of bars on chart.
`code_page()`		Returns the integer value representing the current code page (character encoding) used by the MetaTrader 5 terminal.
`ping_last()`		Returns the last recorded ping time (in microseconds) between the MetaTrader terminal and the broker's server.
`community_balance()`		Returns the current balance of the user's MQL5 community account.
`retransmission()`		Returns the rate of data retransmission from the server to the terminal.

Example usage

```
import MetaTrader5 as mt5
from Trade.TerminalInfo import CTerminalInfo


if not mt5.initialize(r"c:\Users\Omega Joctan\AppData\Roaming\Pepperstone MetaTrader 5\terminal64.exe"):
    print("Failed to initialize Metatrader5 Error = ",mt5.last_error())
    quit()

terminal = CTerminalInfo()

print(f"""
Terminal Information

--- String type ---

Name: {terminal.name()}
Company: {terminal.company()}
Language: {terminal.language()}
Terminal Path: {terminal.path()}
Data Path: {terminal.data_path()}
Common Data Path: {terminal.common_data_path()}

--- Integers type ---

Build: {terminal.build()}
Connected: {terminal.is_connected()}
DLLs Allowed: {terminal.is_dlls_allowed()}
Trade Allowed: {terminal.is_trade_allowed()}
Email Enabled: {terminal.is_email_enabled()}
FTP Enabled: {terminal.is_ftp_enabled()}
Notifications Enabled: {terminal.are_notifications_enabled()}
Community Account: {terminal.is_community_account()}
Community Connected: {terminal.is_community_connection()}
MQID: {terminal.is_mqid()}
Trade API Disabled: {terminal.is_tradeapi_disabled()}
Max Bars: {terminal.max_bars()}
Code Page: {terminal.code_page()}
Ping Last (µs): {terminal.ping_last()}
Community Balance: {terminal.community_balance()}
Retransmission Rate: {terminal.retransmission()}
""")

mt5.shutdown()
```

Outputs

```
Terminal Information

--- String type ---

Name: Pepperstone MetaTrader 5
```

```
Company: Pepperstone Group Limited
Language: English
Terminal Path: c:\Users\Omega Joctan\AppData\Roaming\Pepperstone MetaTrader 5
Data Path: C:\Users\Omega Joctan\AppData\Roaming\MetaQuotes\Terminal\F4F6C6D7A7155578A6DEA66D12B1D40D
Common Data Path: C:\Users\Omega Joctan\AppData\Roaming\MetaQuotes\Terminal\Common

--- Integers type ---

Build: 4755
Connected: True
DLLs Allowed: True
Trade Allowed: True
Email Enabled: True
FTP Enabled: False
Notifications Enabled: False
Community Account: True
Community Connected: True
MQID: False
Trade API Disabled: False
Max Bars: 100000000
Code Page: 0
Ping Last (µs): 251410
Community Balance: 900.026643
Retransmission Rate: 0.535847326494355
```

The CTrade Class

This class provides easy access to the trade functions. Unlike the previous classes which return the information about symbols, MetaTrader 5 terminal, historical deals, and the account, this function is what we need for opening trades.

Setting parameters

Instead of setting parameters such as Magic Number, filling type, and deviation value in points using seprate functions like in CTrade MQL5 class, in our Python class let's configure all of those in a class constructor.

```
class CTrade:

    def __init__(self, magic_number: int, filling_type_symbol: str, deviation_points: int):
```

This reduces the room for errors as calling seprate functions could be forgotten, hence leading to runtime errors which could occur due to empty and none values.

Python custom CTrade class	MQL5 built-in CTrade class	
Operation with orders		
`order_open(self, symbol: str, volume: float, order_type: int, price: float, sl: float = 0.0, tp: float = 0.0, type_time: int = mt5.ORDER_TIME_GTC, expiration: datetime = None, comment: str = "") -> bool`	OrderOpen	Places a pending order with specified parameters.
`order_modify(self, ticket: int, price: float, sl: float, tp: float, type_time: int = mt5.ORDER_TIME_GTC, expiration: datetime = None, stoplimit: float = 0.0) -> bool:`	OrderModify	modifies the pending order with specified parameters.
`order_delete(self, ticket: int) -> bool`	OrderDelete	Deletes a pending order.
Operations with positions		
`position_open(self, symbol: str, volume: float, order_type: int, price: float, sl: float, tp: float, comment: str="") -> bool`	PositionOpen	Opens a position with specified parameters.
`position_modify(self, ticket: int, sl: float, tp: float) -> bool`	PositionModify	Modifies position parameters by the

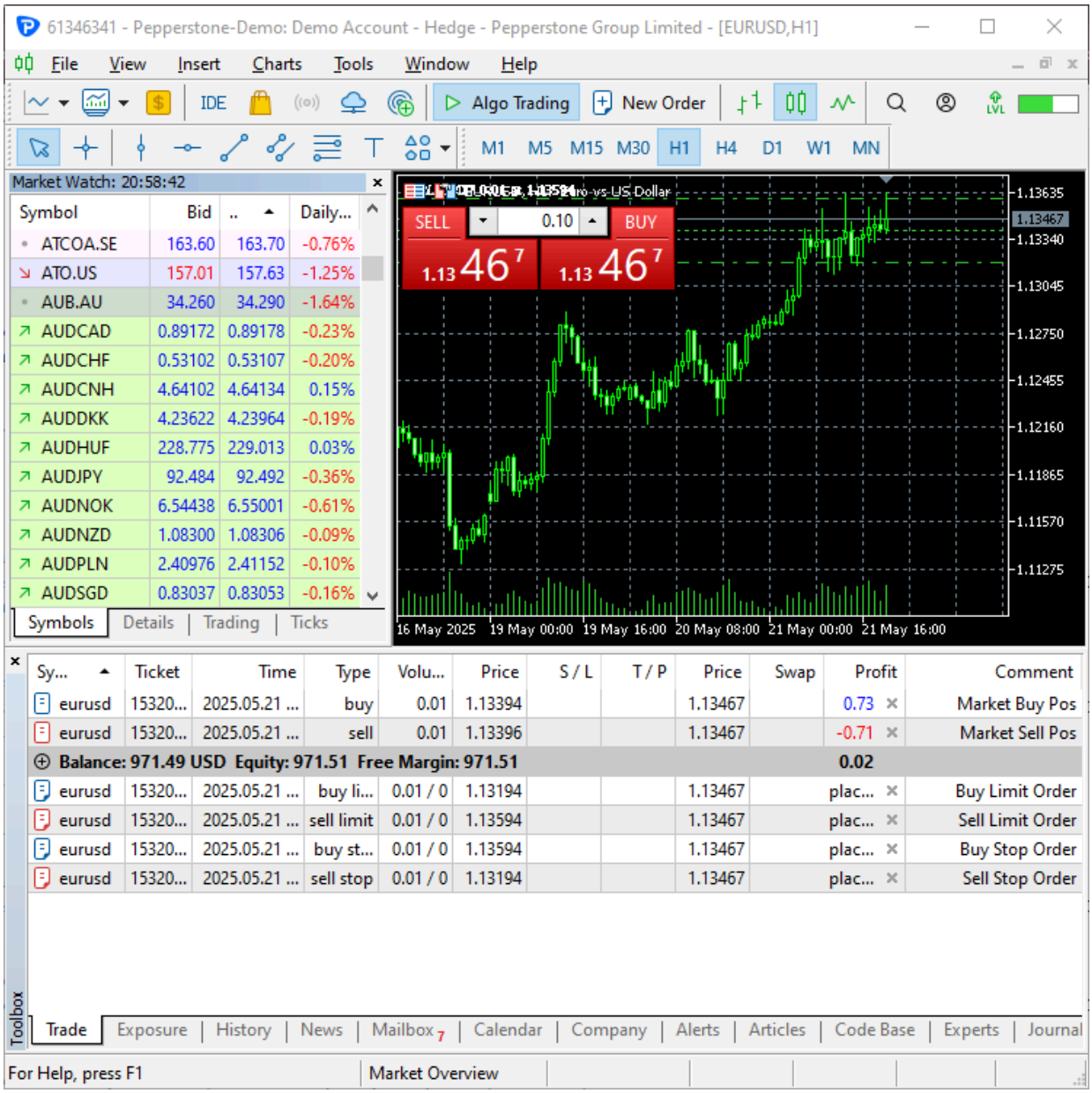
Python custom CTrade class	MQL5 built-in CTrade class	
		specified ticket.
`position_close(self, ticket: int, deviation: float=float("nan")) -> bool`	PositionClose	Closes a position for the specified symbol.
Additional methods		
`buy(self, volume: float, symbol: str, price: float, sl: float=0.0, tp: float=0.0, comment: str="") -> bool`	Buy	Opens a long position with the specified parameters
`sell(self, volume: float, symbol: str, price: float, sl: float=0.0, tp: float=0.0, comment: str="") -> bool`	Sell	Opens a short position with the specified parameters
`buy_limit(self, volume: float, price: float, symbol: str, sl: float=0.0, tp: float=0.0, type_time: float=mt5.ORDER_TIME_GTC, expiration: datetime=None, comment: str="") -> bool`	BuyLimit	Opens a pending order of the Buy Limit type with specified parameters.
`sell_limit(self, volume: float, price: float, symbol: str, sl: float=0.0, tp: float=0.0, type_time: float=mt5.ORDER_TIME_GTC, expiration: datetime=None, comment: str="") -> bool`	BuyStop	Opens a pending order of the Sell Limit type with specified parameters.
`buy_stop(self, volume: float, price: float, symbol: str, sl: float=0.0, tp: float=0.0, type_time: float=mt5.ORDER_TIME_GTC, expiration: datetime=None, comment: str="") -> bool`	SellLimit	Places a pending order of the Buy Stop type with specified parameters.
`sell_stop(self, volume: float, price: float, symbol: str, sl: float=0.0, tp: float=0.0, type_time: float=mt5.ORDER_TIME_GTC, expiration: datetime=None, comment: str="") -> bool`	SellStop	Places a pending order os the Sell Stop type with specified parameters
`buy_stop_limit(self, volume: float, price: float, symbol: str, sl: float=0.0, tp: float=0.0, type_time: float=mt5.ORDER_TIME_GTC, expiration: datetime=None, comment: str="") -> bool`		Places a pending order of the Buy Stop limit type with specified parameters
`sell_stop_limit(self, volume: float, price: float, symbol: str, sl: float=0.0, tp: float=0.0, type_time: float=mt5.ORDER_TIME_GTC, expiration: datetime=None, comment: str="") -> bool`		Places a pending order of the Sell Stop limit with specified parameters.

Now, let's use the CTrade class in Python to open a couple of positions and pending orders.

```
import MetaTrader5 as mt5
from Trade.Trade import CTrade
from Trade.SymbolInfo import CSymbolInfo
from datetime import datetime, timedelta
```

```
if not mt5.initialize(r"c:\Users\Omega Joctan\AppData\Roaming\Pepperstone MetaTrader 5\terminal64.exe"):
    print("Failed to initialize Metatrader5 Error = ",mt5.last_error())
    quit()

symbol = "EURUSD"

m_symbol = CSymbolInfo(symbol=symbol)
m_trade = CTrade(magic_number=1001,
                 deviation_points=100,
                 filling_type_symbol=symbol)

m_symbol.refresh_rates()

ask = m_symbol.ask()
bid = m_symbol.bid()

lotsize = m_symbol.lots_min()

# === Market Orders ===

m_trade.buy(volume=lotsize, symbol=symbol, price=ask, sl=0.0, tp=0.0, comment="Market Buy Pos")
m_trade.sell(volume=lotsize, symbol=symbol, price=bid, sl=0.0, tp=0.0, comment="Market Sell Pos")

# expiration time for pending orders
expiration_time = datetime.now() + timedelta(minutes=1)

# === Pending Orders ===

# Buy Limit - price below current ask
m_trade.buy_limit(volume=lotsize, symbol=symbol, price=ask - 0.0020, sl=0.0, tp=0.0, type_time=mt5.ORDER_TIME_
                 comment="Buy Limit Order")

# Sell Limit - price above current bid
m_trade.sell_limit(volume=lotsize, symbol=symbol, price=bid + 0.0020, sl=0.0, tp=0.0, type_time=mt5.ORDER_TIM
                   comment="Sell Limit Order")

# Buy Stop - price above current ask
m_trade.buy_stop(volume=lotsize, symbol=symbol, price=ask + 0.0020, sl=0.0, tp=0.0, type_time=mt5.ORDER_TIME_
                 comment="Buy Stop Order")

# Sell Stop - price below current bid
m_trade.sell_stop(volume=lotsize, symbol=symbol, price=bid - 0.0020, sl=0.0, tp=0.0, type_time=mt5.ORDER_TIME_
                  comment="Sell Stop Order")

# Buy Stop Limit - stop price above ask, limit price slightly lower (near it)
m_trade.buy_stop_limit(volume=lotsize, symbol=symbol, price=ask + 0.0020, sl=0.0, tp=0.0, type_time=mt5.ORDER_
                       comment="Buy Stop Limit Order")

# Sell Stop Limit - stop price below bid, limit price slightly higher (near it)
m_trade.sell_stop_limit(volume=lotsize, symbol=symbol, price=bid - 0.0020, sl=0.0, tp=0.0, type_time=mt5.ORDE
                        comment="Sell Stop Limit Order")

mt5.shutdown()
```

Outcomes



Conclusion

The trade classes are one of the good things to happen in MQL5, back in a day we used to write everything from scratch, something which is extremely tiresome and leads to a plenty of bugs as I explained earlier. By extending the MetaTrader 5 python package to libraries (modules) in Python which are very similar in syntax to those in MQL5 it helps developers leverage the knowledge they already have working with MQL5 to Python applications.

These custom Python libraries can help mitigate the Intellisense support issue, by adding the "Docstrings" in your Python functions and classes, text editors such as Visual Studio Code can help document your code and highlight the parameters making the coding process fun and much easier. For example, inside the buy method in the CTrade class there is a short description of the function.

```
class CTrade:
# ....

    def buy(self, volume: float, symbol: str, price: float, sl: float=0.0, tp: float=0.0, comment: str="") ->

        """
        Opens a buy (market) position.

        Args:
            volume: Trade volume (lot size)
            symbol: Trading symbol (e.g., "EURUSD")
            price: Execution price
            sl: Stop loss price (optional, default=0.0)
            tp: Take profit price (optional, default=0.0)
            comment: Position comment (optional, default="")

        Returns:
            bool: True if order was sent successfully, False otherwise
        """
```

This function will now get described in VS Code.

```python
m_trade = CTrade(magic_number=1001,
                 deviation_points=100,
                 filling_type_symbol=symbol)


m_symbol.refresh_rates()

ask = m_symbol.ask()
bid = m_symbol.bid()

lotsize = m_symbol.lots_min()

# === Market Orders ===

m_trade.buy(volume=lotsize, symbol=symbol, price=ask, sl=0.0, tp=0.0, comment="Market Buy Pos")
m_trade.sell(volume=lotsize, symbol=symbol, price=bid, sl=0.0, tp=0.0, comment="Market Sell Pos")


# expiration time for pending orders
expiration_time = datetime.now() + timedelta(minutes=1)

# === Pending Orders ===

# Buy Limit - price below current ask
m_trade.buy_limit(volume=lotsize, symbol=symbol, price=ask - 0.0020, sl=0.0, tp=0.0, type_time=mt5.ORDER_TIME_SPECIFIED, expiration=expiration_time,
                  comment="Buy Limit Order")
```

Ln 40, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.9.0 ('mt5-env': conda)

Simply put, this article serves as the documentation of the trade classes for MetaTrader5 I made in the Python programming language, please let me know your thoughts in the discussion section.

Best regards.

Attachments Table

Filename & Path	Description & Usage
Modules (libraries)	
Trade\AccountInfo.py	Contains the CAccountInfo class
Trade\DealInfo.py	Contains the CDealInfo class
Trade\HistoryOrderInfo.py	Contains the CHistoryOrderInfo class
Trade\OrderInfo.py	Contains the COrderInfo class
Trade\PositionInfo.py	Contains the CPositionInfo class
Trade\SymbolInfo.py	Contains the CSymbolInfo class
Trade\TerminalInfo.py	Contains the CTerminalInfo class
Trade\Trade.py	Contains the CTrade class
Test files	
accountinfo_test.py	A script for testing methods offered by the CAccountInfo class
dealinfo_test.py	A script for testing methods offered by the CDealInfo class
error_description.py	Contains function to describe error and return codes to human-readable strings
historyorderinfo_test.py	A script for testing methods offfered by the CHistoryOrderInfo class
orderinfo_test.py	A script for testing methods offered by the COrderInfo class
positioninfo_test.py	A script for testing methods offered by the CPositionInfo class

Filename & Path	Description & Usage
symbolinfo_test.py	A script for testing methods offered by the CSymbolInfo class
terminalinfo_test.py	A script for testing methods offered by the CTerminal class
main.py	A script for testing the CTrade class, think of it as a final trading robot in Python